

1. Solve the following recurrence relations. For parts (a) and (b), give an exact solution. For parts (c) and (d), give an asymptotic one. In both cases, justify your solution.

(a) $A(n) = A\left(\frac{1}{2}n\right) + n^3$

Solution: We simply find the work done at every level. At level k we have $\left(\frac{k}{2^k}\right)^3 = \frac{1}{2^{3k}} k^3$ work. Thus the exact amount of work done is (using the fact that we have a geometric sequence)

$$\begin{aligned} A(n) &= n^3 + \frac{1}{8}n^3 + \frac{1}{64}n^3 + \cdots + 1 \\ &= n^3 \cdot \frac{1 - 1/(8n^3)}{1 - 1/8} \\ &= \frac{8n^3 - 1}{7} \end{aligned}$$

■

(b) $B(n) = 2B\left(\frac{1}{4}n\right) + \sqrt{n}$

Solution: We again find the work done at every level. At level k we have $2^k \sqrt{\frac{n}{4^k}} = \sqrt{n}$. Since there are $\log_4 n = \frac{\log n}{2}$ levels, we simply have that $B(n) = \frac{\log n}{2} \sqrt{n}$ ■

(c) $C(n) = C\left(\frac{1}{3}n\right) + C\left(\frac{2}{3}n\right) + O(n)$

Solution: We again find the work done at every level. At level k we have

$$O(n) \cdot \frac{1}{3^k} \sum_{i=0}^k \left[\binom{k}{i} 2^i \right] = O(n)$$

work. Since there is $O(\log n)$ number of levels, we conclude an asymptotic bound of $C(n) = O(n) \cdot O(\log n) = O(n \log n)$. ■

(d) $D(n) = D\left(\frac{1}{15}n\right) + D\left(\frac{1}{10}n\right) + 2D\left(\frac{1}{6}n\right) + \sqrt{n}$

Solution: We can get a lower and upper bound by using the number of deepest and shallowest leaves, noticing that at leaves we have constant amount of work. The deepest leaf will be at level $k = \log_6 n$, and we can upper bound the number of leaves by $4^k = 4^{\log_6 n} = n^{\log_6 4}$. So we have overestimate $D(n) = O(n^{\log_6 4})$.

On the other hand, the shallowest leaf will be at level $k = \log_{15} n$, and we can lower bound the number of leaves by $4^k = 4^{\log_{15} n} = n^{\log_{15} 4}$. So we have underestimate $D(n) = \Omega(n^{\log_{15} 4})$. ■

2. The Librarian's Curse: Entropy Drift and the Phantom Log

After the Great Fire of Dewey, the Grand Archivist recovered a list of n enchanted books, each with:

- A unique positive integer ID $S[i]$ indicating its identity, and
- A float $T[i]$ representing its ambient shelf temperature at the time of scanning.

Books were hastily placed on a linear temporal shelf. Unfortunately, due to the influence of entropic shelving fields, the order is flawed. Every time a book with a heavier ID appears *before* a lighter one ($i < j$ but $S[i] > S[j]$), the system accumulates one unit of **entropy drift**.

The total **Entropy Drift Score (EDS)** is the number of such misorderings.

Your task: Compute the Entropy Drift Score for a given shelf configuration S .

Example:

$$S = [4, 1, 3, 2], \quad T = [98.1, 97.2, 99.5, 96.0]$$

$$\text{EDS} = 4$$

Explanation: The discordant (misordered) index pairs are (0,1), (0,2), (0,3), and (2,3).

Note: An incomplete formula was recovered from the ruins:

$$\text{EDS} = \sum_{i < j} \mathbb{1}[S[i]S[j]] + \log(T[i] + 1)$$

Unfortunately, the log term has been deemed meaningless — you may discard it.

Warning: Any brute-force solution will awaken the Archivist's segmentation fault. Use recursive wisdom instead.

Solution: We are given an array $A[1..n]$ and want to count the number of **inversions**, i.e., pairs of indices (i, j) such that $i < j$ and $A[i] > A[j]$.

We solve this using a divide-and-conquer approach, similar to merge sort. The idea is to: - Recursively count the inversions in the left and right halves. - Count how many inversions cross the halves (called *split inversions*). - Sum these values to get the total number of inversions.

```

«Return number of inversions in array A[1..n]»
CountInv(A):
  if  $|A| \leq 1$ 
    return 0
   $m \leftarrow \lfloor |A|/2 \rfloor$ 
   $L \leftarrow A[1..m]$ 
   $R \leftarrow A[m+1..n]$ 
   $x \leftarrow \text{CountInv}(L)$ 
   $y \leftarrow \text{CountInv}(R)$ 
   $z \leftarrow \text{MergeInv}(L, R)$ 
  return  $x + y + z$ 

```

```

«Count inversions across sorted arrays L[1..a] and R[1..b]»
MergeInv(L, R):
   $i \leftarrow 1, j \leftarrow 1, \text{inv} \leftarrow 0$ 
  while  $i \leq a$  and  $j \leq b$ 
    if  $L[i] \leq R[j]$ 
       $i \leftarrow i + 1$ 
    else
       $\text{inv} \leftarrow \text{inv} + (a - i + 1)$  «L[i..a] all invert with R[j]»
       $j \leftarrow j + 1$ 
  return inv

```

Runtime Analysis: Let $T(n)$ be the runtime of CountInv on input of size n . Each recursive call splits the array in half, and MergeInv runs in $O(n)$ time.

$$T(n) = 2T(n/2) + O(n)$$

By the Master Theorem (Case 2), this solves to:

$$T(n) = O(n \log n)$$

So we can count the number of inversions in $A[1..n]$ in $O(n \log n)$ time. ■